Database Design — The Primer

database-design-primer.txt

```
================================================================================
  HOW TO DESIGN A DATABASE  -  a from-scratch primer on ER design
  For: hsjrua        Written: 2026-09-29        By: your tutor
  Companion to the student-panel materials - but this one is GENERAL.
================================================================================
```

This primer teaches the CRAFT of designing a database: how you get from "here is a situation in the real world" to "here are the tables, and here is why they look like this." It uses small, everyday examples - a library, a school, a shop - on purpose. student-panel is an advanced, finished design; you will design far better once you have built the method here first, on things small enough to hold in your head.

Nothing here asks you to touch a computer. Design happens on paper, in your head, in conversation. Read slowly. Keep a pencil and a blank sheet beside you.

A NOTE ON THIS EDITION. This is the fully-drawn edition. Wherever an idea can be shown as a table of real rows or as a diagram, it is drawn, not just described - because you learn design by SEEING data sit in its places and by seeing lines connect boxes. Everything is hand-drawn in plain characters. Read this file in a MONOSPACE (fixed-width) font - the one your code editor uses - or the tables and diagrams will not line up and the crow's-foot marks will drift. If a box looks crooked, your font is proportional; switch it. Nothing here is decorative; every table and every diagram is there to be read, row by row and mark by mark.

```
--------------------------------------------------------------------------------
  PART 0 - HOW TO USE THIS PRIMER
--------------------------------------------------------------------------------
```

WHAT THIS IS
  A complete beginner's course in database DESIGN and, at its heart, in ER ("entity-relationship") modelling - the standard way people think about and draw a database before they build it. We go biggest-picture first, then descend from major ideas to minor details, and we do that descent again inside each part. Before every piece we pause on four questions: what is it, how does it work, what could (and could not) be done with it, and should it exist at all.

WHAT THIS IS NOT
  It is not about student-panel yet, and it is not a SQL syntax manual. You will meet a little SQL near the end, only to show how a design becomes real tables. The thinking comes first; the typing comes much later.

THE CALLOUTS (watch for these)
  >> CONTEMPLATE - stop reading. Think, or better, sketch, before you read on.
                   Your own answer first; mine second. This is the real work.
  TRY IT   - a small design to attempt on your blank sheet.
  KEY IDEA   - the one sentence from a section worth memorising.
  >> PITFALL   - a mistake almost every beginner makes here. Now you won't.

THE SIGNPOSTS AT A GLANCE

signpost	what it asks of you	what to do with it
>> CONTEMPLATE	stop and think / sketch first	write YOUR answer, then

```
  |                |                                | read mine; compare      |
  | TRY IT         | a tiny paper exercise          | actually do it on paper |
  | KEY IDEA       | the one-sentence takeaway      | memorise this sentence  |
  | >> PITFALL     | a named common mistake         | notice when you slip in |
  +----------------+--------------------------------+-------------------------+
```

A MAP OF THE WHOLE PRIMER (so you always know where you are)

Each Part teaches one layer and leaves you one artifact you can draw by hand.

part	what it teaches	the artifact you can draw after
0	how to read this file	(this map)
1	what design IS; the big picture	a list of QUESTIONS a system answers
2	entities, attributes, links	a list of nouns/adjectives/verbs
3	keys (identity + pointers)	a chosen primary key per table
4	reading/drawing ER diagrams	a crow's-foot diagram
5	the five hard patterns	a bridge/tree/ternary resolved
6	normalisation (one fact, once)	tables split to 3NF
7	the 10-step method	a filled-in recipe
8	a full worked example (library)	final tables with sample rows
9	judgement (the grey areas)	a decision with a stated reason
10	practice	your own designs

Other files you have refer back to these Part numbers (e.g. "primer Part 5"), so the numbering here is fixed - treat it like street addresses.

HOW TO READ IT

Once, straight through, to see the shape. Then again, slowly, doing the CONTEMPLATE and TRY IT prompts for real. When you come back, bring your sketches and your disagreements - disagreeing well is how designers are made.

KEY IDEA: read biggest-picture first. What and why before how. Every drawn table and diagram below is there so you can SEE the idea, not just be told it.

PART 1 - WHAT "DESIGNING A DATABASE" ACTUALLY MEANS (the biggest picture)

Start as high up as we can. A database is a place that remembers facts for you, in an organised way, so a program can store them and ask questions of them later. Designing one means deciding WHAT facts you will keep and HOW they are arranged. That is it. Everything technical below serves those two decisions.

Here is the mental move at the centre of all of it: a database is a small, deliberate MODEL of a slice of the real world. The real world is infinitely detailed; your model keeps only the parts your program cares about and throws the rest away. A library's database "knows" a book has a title and an author, but not that its cover is scuffed or that it smells of coffee. Design is the art of choosing what to keep.

WHY NOT JUST ONE BIG SPREADSHEET?

Because facts repeat, and repetition rots. Put a member's phone number next to every book they borrow, and the day they change numbers you must find and fix it in twenty places - miss one and your data now disagrees with itself. A good design stores each fact ONCE, in the one place it belongs, and links to it from everywhere else. Most of this primer is really about that single sentence.

Here is the rot, drawn. One flat sheet, one row per borrowing, the member's details copied onto every row:

```
+---------+-------------+--------------+-----------+------------+
| loan_id | member_name | member_phone | book      | due_date   |
+---------+-------------+--------------+-----------+------------+
| 1       | Sara Adib   | 555-1020     | Dune      | 2026-10-14 |
| 2       | Sara Adib   | 555-1020     | Emma      | 2026-10-14 |
| 3       | Omar Reyes  | 555-7788     | Dune      | 2026-10-20 |
| 4       | Sara Adib   | 555-9999 (!) | The Hobbit| 2026-10-28 |
+---------+-------------+--------------+-----------+------------+
```

Look at row 4: someone updated Sara's phone there but not on rows 1 and 2. Now the sheet claims Sara has TWO different phone numbers and cannot say which is right. Nobody lied; the shape of the sheet made the mistake possible. The whole of Part 6 is about making that mistake impossible, by storing Sara's phone in exactly one place and pointing at it. For now just feel the ache of row 4.

THE THREE LEVELS OF A DESIGN

Designers deliberately think at three heights, from ideas down to storage:

1. CONCEPTUAL - the ideas and their connections, in plain language, with no thought of any particular database product. "A member borrows many copies; a copy belongs to one book." This is where ER modelling lives.
2. LOGICAL - those ideas turned into tables, columns, keys and links, still without worrying which database engine runs it.
3. PHYSICAL - the real thing on a real engine: exact data types, indexes, storage. This is last, and it is the least interesting part of design.

We spend almost all our time at levels 1 and 2. Beginners rush to level 3 (typing commands); good designers linger at level 1 (getting the ideas right).

The same idea, shown descending through the three heights:

```
+------------+--------------------------------+---------------------------------+
| level      | what you say                   | library example                 |
+------------+--------------------------------+---------------------------------+
| conceptual | nouns + verbs, plain words     | "a member borrows copies"       |
| logical    | tables, columns, keys, links   | loan(loan_id, member_id FK,     |
|            |                                | copy_id FK, borrowed_on)        |
| physical   | types, sizes, indexes, engine  | loan_id BIGINT, index on        |
|            |                                | member_id, Postgres             |
+------------+--------------------------------+---------------------------------+
```

WHERE "ER" COMES FROM

In 1976 Peter Chen proposed describing data as ENTITIES (things) joined by RELATIONSHIPS (how the things relate). It stuck because it matches how people already talk: nouns and verbs. An ER diagram is just a picture of nouns and verbs. You already think this way; we are only making it precise.

KEY IDEA: a database is a chosen model of reality; design is deciding what to keep and how to arrange it so every fact lives in exactly one place.

WHAT MAKES A DESIGN "GOOD"?

Four things, roughly in order of importance:

- FAITHFUL: it can represent every real situation you care about, and it forbids situations that can't really happen (a loan with no book).
- NON-REDUNDANT: each fact is stored once, so it can never contradict itself.
- ANSWERABLE: the questions you will actually ask are easy and fast to ask.
- SIMPLE: no table, column, or link exists without a reason you can state.

Notice these can pull against each other - chasing zero redundancy can make questions harder to ask. Design is the judgement that balances them. There is rarely one "right" schema, only better and worse trade-offs for a purpose.

Keep this scorecard beside you and test any design against it:

property	the test question	failing symptom
faithful	can it store every real case, and refuse impossible ones?	you can't record a true situation; or you can store a nonsense one
non-redundant	is each fact in exactly 1 place	the same fact in two rows can disagree (row 4 above)
answerable	are my real questions easy?	a common question needs a monstrous query
simple	can I name a reason for each table and column?	a table nobody can explain

BEGIN FROM THE QUESTIONS, NOT THE TABLES

The single most useful habit: before drawing anything, write down the questions the system must answer and the actions it must support. "Which books is this member holding right now?" "How many copies of this title do we own?" "Who had this book before it was returned damaged?" A design is good exactly to the degree it answers ITS questions well. Tables you can't tie back to a real question are usually tables that shouldn't exist.

>> CONTEMPLATE: imagine a tiny system for a coffee shop's loyalty cards. Before reading on, write five questions it must answer and two actions it must support. Keep the list - we test designs against lists like this.

A WORKED VERSION (peek only after you have written your own).

Here is one reasonable answer, written the way Step 1 of the method (Part 7) wants it. Yours will differ - that is fine; the point is the HABIT.

#	QUESTION the loyalty system must answer
Q1	how many stars does customer C have right now?
Q2	which rewards has C already redeemed?
Q3	how many stars were earned shop-wide last week?
Q4	is this card number valid and whose is it?
Q5	which customers have not visited in 60 days?

#	ACTION it must support
A1	record a visit that earns stars
A2	redeem stars for a reward

Notice already: Q1's "stars right now" smells DERIVED (sum of stars earned minus stars spent) rather than a number you store and must keep fresh. Hold that thought - it returns in Part 2.2 and Part 9. Good question lists leak design hints like this before you have drawn a single box.

>> PITFALL: starting by asking "what columns do I need?" That is level-3 thinking wearing a disguise. Start at level 1: what THINGS exist, and how do they relate? Columns come later and almost fall out on their own.

PART 2 - THE THREE PRIMITIVES: ENTITIES, ATTRIBUTES, RELATIONSHIPS

Almost every ER model is built from just three kinds of thing. Learn these three

deeply and the rest of design is refinement. We take them major-to-minor.

2.1 ENTITIES - the "things"

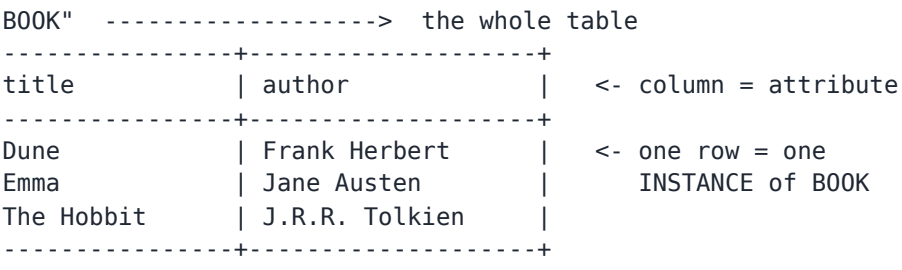
An entity is a thing your system needs to remember: a Member, a Book, an Order, a Student, a Room. If you can point at it and say "we keep facts about that," it is a candidate entity. In speech, entities are usually the NOUNS.

Two words you must not confuse:

- ENTITY TYPE (or entity set): the KIND of thing - "Book". This becomes a TABLE. When people say "entity" they usually mean this.
- ENTITY INSTANCE: one particular one - "Book #4417, 'Dune'". This becomes a ROW in that table.

A table is the type; each row is an instance. Hold that distinction; a lot of confusion later is really this confusion.

Drawn, the type IS the table and each instance IS one row of it:



"Book" (the kind) is the table. "Book #4417, Dune" (one of them) is a row.

WHAT IS *NOT* AN ENTITY (yet)

- A mere property of something else ("a book's title") - that is an ATTRIBUTE, not an entity of its own... usually. See the contemplation below; the line moves.
- An action ("borrowing") - that is usually a RELATIONSHIP. But actions worth remembering in detail often become entities too (a "Loan"). Judgement again.

A quick test you can run on any noun you underline. Ask: do we keep facts ABOUT this thing itself? If yes, it is an entity; if it is just one fact about another thing, it is an attribute.

candidate noun	for this system...	entity or attr?	why
Book	library: lends books	ENTITY (table)	many facts: it has title,isbn
title	library	attribute	one fact OF a book
Publisher	library: tracks who prints each title	ENTITY (table)	has name,city, many books
colour	car-rental app	attribute	one fact OF a car
Colour	paint maker: formulas, suppliers per colour	ENTITY (table)	has code, formula, etc.
borrowing	library	RELATIONSHIP -> becomes Loan	a verb joining member & copy

>> CONTEMPLATE: is COLOUR an entity or an attribute? For a car-rental app, colour is probably just an attribute of Car. For a paint manufacturer, Colour is a full entity with a code, a formula, a supplier. Same word, different model - because the PURPOSE differs. What decides it? Ask: do we keep facts ABOUT the colour itself? If yes, it wants to be an entity.

TRY IT: for a cinema, underline the nouns in "a customer books seats for a screening of a film." Which become entities (we keep facts about them), and which are just attributes? Which word is a verb hiding a future table?

KEY IDEA: entity type = table = a kind of thing; entity instance = row = one specific thing. Entities are the nouns you keep facts about.

2.2 ATTRIBUTES - what we know about a thing

An attribute is a fact stored about an entity: a Member's name, a Book's title, a Student's birthdate. In tables, attributes become COLUMNS. Choosing good attributes is choosing what your model notices about the world.

Attributes come in kinds worth naming, because each changes how you model it:

- ATOMIC (simple) vs COMPOSITE. Atomic means "one indivisible value" - a price, a title. Composite means it is really several values wearing one name - "address" is street + city + postcode. Composites usually get split into separate columns, because you will want to search or sort by the parts.

- SINGLE-VALUED vs MULTI-VALUED. A person has one birthdate (single) but may have many phone numbers (multi). This one matters enormously: a table cell holds ONE value, so a multi-valued attribute CANNOT simply be a column. It almost always signals a new entity or a link table. Remember this; it is the seed of the most common design pattern of all (Part 5).

- STORED vs DERIVED. Some facts are stored directly (birthdate); others can be COMPUTED from stored ones (age = today - birthdate). Derived facts are best NOT stored - you recompute them when asked, so they can never drift out of date. "Derive, don't store" is a rule you will meet again and again. (There is one deliberate exception - a value you FREEZE on purpose at a moment in time, like the price charged on a sale. We meet it in Part 9.)

- KEY attributes - the ones that identify the thing. Important enough to get their own part (Part 3).

The four kinds on one card, with the modelling move each one forces:

kind	example	what it forces you to do
atomic	price, title, isbn	one plain column
composite	address, full name	split into columns (street, city...)
single-valued	birthdate	one plain column
multi-valued	phone numbers	its OWN rows/table (never phone1,2,3)
stored	birthdate	keep it
derived	age, order total	compute on demand; do NOT store

COMPOSITE, DRAWN: split the parts you will search or sort by.

BAD (one lump you cannot sort by city):

memb_id	address
M-1001	14 Rose Lane, Brighton, BN1 2AA
M-1002	7 Oak Street, Leeds, LS1 4DF

BETTER (sort by city, post to a postcode, all easy):

```
  | memb_id  | street        | city      | postcode  |
  +----------+---------------+-----------+-----------+
  | M-1001   | 14 Rose Lane  | Brighton  | BN1 2AA   |
  | M-1002   | 7 Oak Street  | Leeds     | LS1 4DF   |
  +----------+---------------+-----------+-----------+

MULTI-VALUED, DRAWN: the classic beginner trap and its cure.
  BAD (fixed columns - what about a 4th phone? and most cells sit empty):
  +----------+-------------+-----------+-----------+-----------+
  | memb_id  | name        | phone1    | phone2    | phone3    |
  +----------+-------------+-----------+-----------+-----------+
  | M-1001   | Sara Adib   | 555-1020  | 555-3344  | (null)    |
  | M-1002   | Omar Reyes  | 555-7788  | (null)    | (null)    |
  +----------+-------------+-----------+-----------+-----------+
  CURE (one row per phone, in its own table, linked back by memb_id):
  +------------+----------+-----------+-----------+
  | phone_id   | memb_id  | number    | kind      |
  +------------+----------+-----------+-----------+
  | P-1        | M-1001   | 555-1020  | mobile    |
  | P-2        | M-1001   | 555-3344  | work      |
  | P-3        | M-1002   | 555-7788  | mobile    |
  +------------+----------+-----------+-----------+
  Now a member can have one phone or ten; nothing sits empty; and you never
  guessed a maximum. This little table is your first glimpse of Part 5.


DERIVED, DRAWN: store the birthdate, NOT the age.
  You store only the left column. "age" is shown here greyed in parentheses to
  mean "computed when asked, never written down":
  +----------+-------------+-------------+----------------------------+
  | memb_id  | name        | birthdate   | (age - DERIVED, not stored)|
  +----------+-------------+-------------+----------------------------+
  | M-1001   | Sara Adib   | 1998-03-11  | (28, recompute each year)  |
  | M-1002   | Omar Reyes  | 2001-07-02  | (25, recompute each year)  |
  +----------+-------------+-------------+----------------------------+
  If you stored "age = 28" it would be a lie by Sara's next birthday and nobody
  would remember to fix it. The birthdate never goes stale; the age always can.
  So: keep the stable fact, compute the changing one. (The exception - a number
  you deliberately FREEZE at a moment, like a sale price - is in Part 9.)

>> CONTEMPLATE: "full name" - atomic or composite? If you will ever sort by
   surname or greet someone by first name, it is composite: store the parts.
   If you truly only ever print it whole, atomic is fine. The USE decides.

>> PITFALL: a column holding "555-1234, 555-9988" (two phone numbers jammed in
   one cell). That is a multi-valued attribute in disguise, and it breaks the
   first rule of tidy tables (Part 6). One value per cell, always.

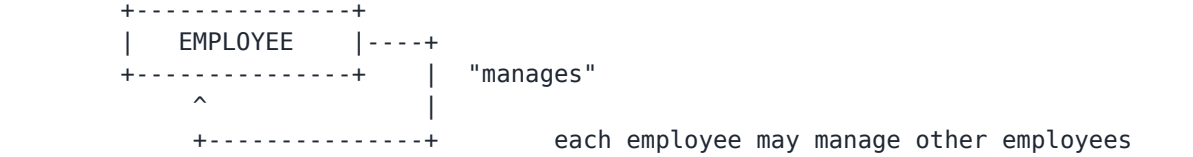

2.3  RELATIONSHIPS - how the things connect
-------------------------------------------
A relationship is an association between entities: a Member BORROWS a Copy; a
Book HAS a Publisher; a Student ENROLS IN a Course. In speech, relationships are
the VERBS joining your nouns.

A relationship's DEGREE is how many entity types it joins:
  - UNARY (recursive): an entity related to itself. An Employee MANAGES another
    Employee; a Category is the PARENT OF another Category. This is how trees and
    hierarchies are built, and it looks strange the first time - one box with a
    line looping back to itself. (Part 5 returns to it; student-panel leans on
    it heavily.)
```

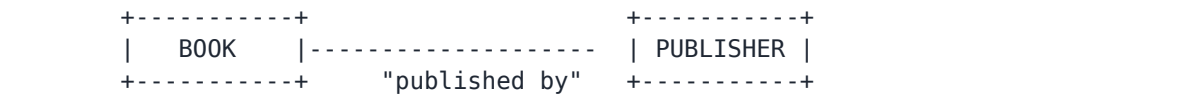
- BINARY: two entity types - by far the most common. Member-Loan, Book-Author.
- TERNARY (and higher): three at once. "A Doctor prescribes a Drug to a Patient" links three things in one fact. These are rarer and usually get re-expressed as an entity in the middle (Part 5).

The three degrees, drawn:

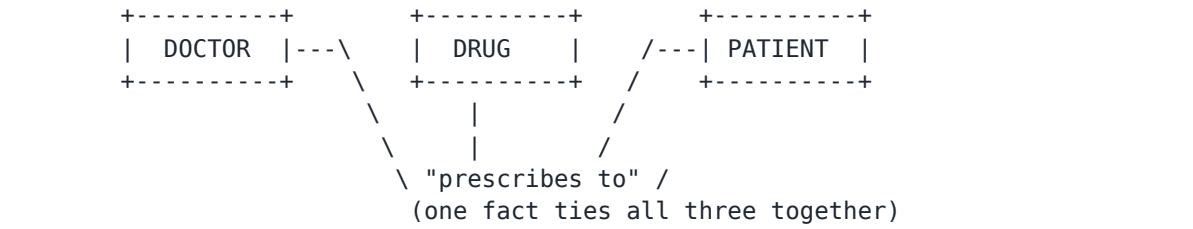
UNARY (one box, line loops back to itself):



BINARY (the everyday case - two boxes, one line):

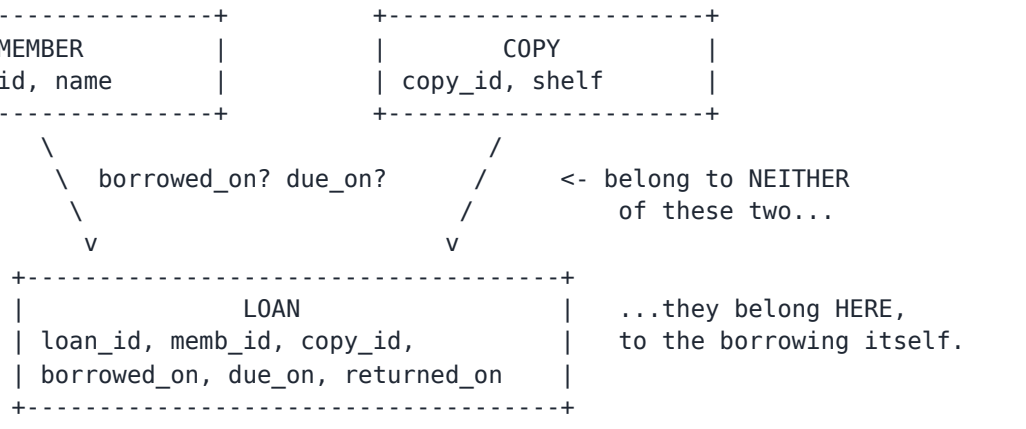


TERNARY (three boxes meeting at one fact - usually turned into a box, Part 5):



Relationships can carry attributes of their OWN. "Member borrows Copy" - but the borrow DATE and the due DATE belong to neither the member nor the copy; they belong to the borrowing itself. A relationship with its own facts is a strong hint that the relationship wants to become an entity (an associative entity - here, a "Loan"). Spotting this early saves redesigns later.

WHOSE FACT IS IT? drawn. The dates fit neither parent; they live on the Loan:



loan_id	memb_id	copy_id	borrowed_on	due_on	returned_on
L-1	M-1001	C-2	2026-09-30	2026-10-14	(null=out)
L-2	M-1002	C-5	2026-09-28	2026-10-12	2026-10-03

returned_on is NULL on L-1 - its emptiness MEANS "still out" (Part 9).

>> CONTEMPLATE: "A Student has a Grade in a Course." Whose fact is the grade? Not the student alone (they have many), not the course alone (it gives many). It belongs to the student-and-course PAIR. Where, then, must it be stored?

KEY IDEA: entities are nouns, attributes are their adjectives, relationships are
the verbs - and verbs can have facts of their own.

2.4 CARDINALITY - the most important question about any relationship

For every relationship you must answer: how many of each side relate to how
many of the other? This is CARDINALITY, and it decides the whole table layout.

Ask it in BOTH directions, one side at a time, in plain English:
  "One member borrows how many copies?" -> many.
  "One copy is borrowed by how many members (at once)?" -> one.
That pair of answers ("many" / "one") makes this a ONE-TO-MANY relationship.

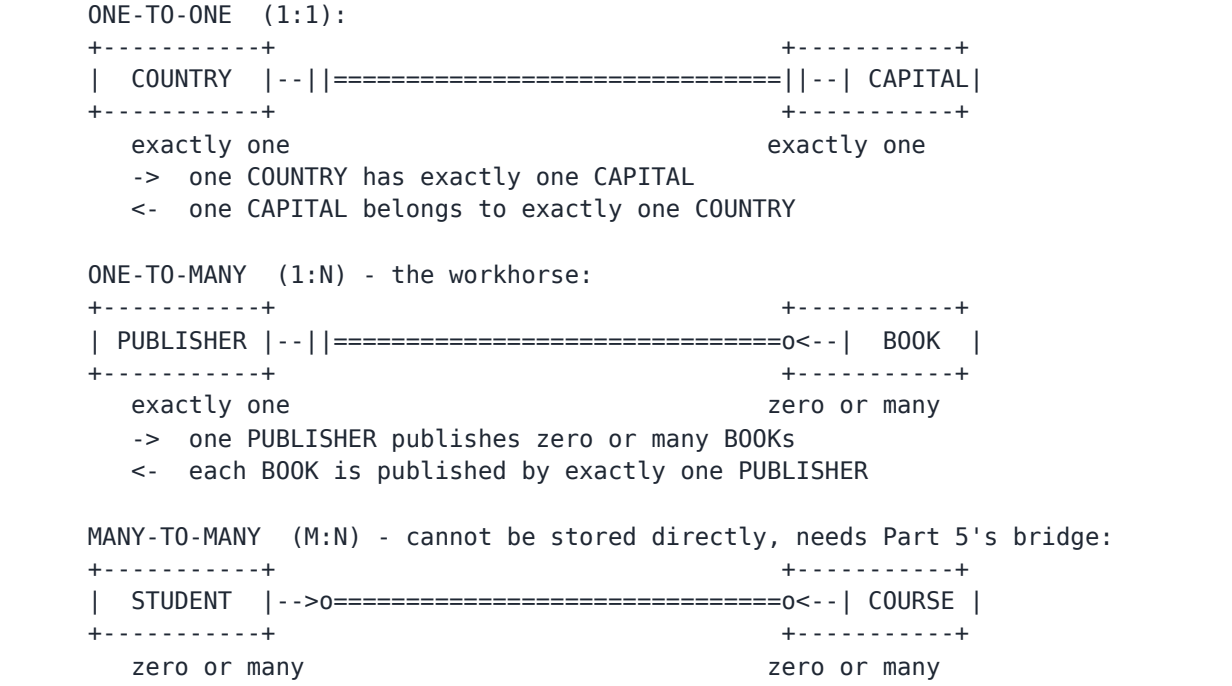
The habit of asking BOTH directions, as a table you can fill in for any link:

relationship	"one LEFT relates to how many RIGHT?"	"one RIGHT relates to how many LEFT?"	type
Country-Capital	one	one	1:1
Publisher-Book	many	one	1:N
Student-Course	many	many	M:N
Member-Loan	many	one	1:N

Fill both middle columns in words, and the last column names itself.

The three shapes you will meet, with a test question for each:
- ONE-TO-ONE (1:1). One country has one capital city; one capital belongs to one country. Rare, and often a hint the two entities could be one - or are split on purpose (optional or sensitive data kept apart).
- ONE-TO-MANY (1:N). One publisher prints many books; each book has one publisher. The workhorse of database design; most links are these.
- MANY-TO-MANY (M:N). One student takes many courses; one course has many students. These CANNOT be stored directly and always need a helper table - the single most important pattern in Part 5.

The three shapes, drawn in crow's foot (how to read the marks is Part 4; for
now just see the shape). Marks nearest a box = the MAXIMUM on that side:



```
      ->  one STUDENT takes zero or many COURSEs
      <-  one COURSE holds zero or many STUDENTs
```

PARTICIPATION (the "minimum" side of cardinality)
Cardinality has a maximum (one/many) AND a minimum (must there be at least one, or is zero allowed?). "Must every loan have a book?" - yes, MANDATORY. "Must every member have a loan?" - no, a new member has zero, OPTIONAL. The minimum is what NULL and "optional" are really about, and it is where a design says which situations are allowed to exist at all.

So each end of a relationship carries a (minimum, maximum) pair. Four little symbols in the next part encode exactly these pairs. Get cardinality right and the diagram almost draws itself; get it wrong and no amount of later cleverness saves the design.

The (min, max) pair for every end, laid out so you can see all four at once:

minimum	maximum	plain words	real example (an end of a link)
0 (opt)	1	zero or one	a person's optional spouse record
1 (mand)	1	exactly one	a loan's book (must have one)
0 (opt)	many	zero or many	a member's loans (new member has 0)
1 (mand)	many	one or many	an invoice's lines (at least one)

>> CONTEMPLATE: for your coffee-shop loyalty system - one card belongs to how many customers? one customer holds how many cards? Is a customer allowed to exist with zero cards? Your answers ARE the design of that link.

TRY IT: write the (min, max) pair for BOTH ends of "a department employs staff." Must a department have at least one employee? Can a brand-new, not-yet-staffed department exist? Can an employee belong to no department? Your three answers decide whether the FK may ever be NULL (Part 3).

PART 3 - KEYS: HOW A DATABASE KNOWS "WHICH ONE"

Two rows that look alike are a disaster waiting to happen. Keys are how a database tells things apart and how one table points reliably at a row in another. Identity is the quiet foundation everything else stands on.

CANDIDATE KEYS - any attribute (or set of attributes) whose value is unique for every row and never empty could serve to identify the row. A member might have several: their membership number, and perhaps their email. Each is a CANDIDATE.

Drawn: in this member table, THREE columns are each unique and never empty - so there are three candidate keys. Any one COULD be the identifier:

memb_id	memb_no	email	name
M-1001	100244	sara@example.org	Sara Adib
M-1002	100245	omar@example.org	Omar Reyes
M-1003	100246	lee@example.org	Mina Lee

```
   ^candidate   ^candidate      ^candidate          not unique
   (surrogate)  (natural)       (natural, but        (two people
                                 changes!)            can share a name)
```

name is NOT a candidate: two members could be "Sara Adib." The other three

each uniquely pick one row - they are the candidates we choose a PK from.

PRIMARY KEY - you pick ONE candidate to be the official identifier. Every table gets exactly one primary key; it is unique, never NULL, and ideally never changes. Everything that points at this row will point using this value.

WHAT MAKES A GOOD PRIMARY KEY? It should be:
- UNIQUE (by definition), and guaranteed so, not just "probably".
- STABLE - it should never need to change. (This rules out a lot of tempting choices, see below.)
- MINIMAL and MEANINGLESS is often best - the less real-world baggage it carries, the less reason it will ever have to change.

Score the tempting candidates against the three rules - this is why we lean to a made-up id:

candidate	unique?	stable?	meaning?	verdict as primary key
name	no	no	yes	never (repeats, changes)
email	yes-ish	no	yes	risky (people change email)
memb_no	yes	mostly	yes	ok natural key; keep UNIQUE
made-up id (surrogate)	yes	YES	no	best PK; add a unique natural column for humans

>> CONTEMPLATE: should a person's name be the primary key? Names repeat (two John Smiths), names change (marriage), names have no guaranteed format. Every property of a good key is violated. So - almost never. What about email? Unique-ish, but people change emails, and then everything pointing at them breaks. This tension leads straight to the next idea.

NATURAL vs SURROGATE KEYS - a real design fork

A NATURAL key is a real-world value that already identifies the thing: an ISBN for a book, a country code, a membership number printed on the card.
A SURROGATE key is a meaningless value the system invents purely to identify the row: an auto-incrementing number (1, 2, 3...) or a random UUID like 3f2a...-.... It means nothing outside the database.

Both an auto-number AND a UUID are surrogate keys - that is not a contradiction. "Surrogate" means "invented, meaningless"; it does not mean "a number." The two just make the meaningless id in different ways:

surrogate kind	looks like	trade-off
auto-number	1, 2, 3, 4 ...	tiny, readable; but reveals row counts and collides if you merge two databases
UUID	3f2a9c7e-...-b1	bulky, ugly; but unique across every machine, leaks nothing, safe to generate anywhere

Trade-offs:
- Natural keys need no extra column and read meaningfully, BUT they change (and then every reference must change too), and "guaranteed unique forever" is rarer than it looks.
- Surrogate keys never change and are always clean, BUT they carry no meaning, so you still keep the natural value as an ordinary (indexed, unique) column for humans to search by.

Most modern designs use a surrogate primary key (auto-number or UUID) AND keep the natural identifier as a separate UNIQUE column - best of both. student-panel does exactly this: UUID primary keys, plus a human "student_code" kept unique.

THE MODERN DEFAULT, DRAWN (student-panel's own pattern): a UUID primary key PLUS a human-readable student_code under a UNIQUE constraint:

id (UUID, PK)	student_code (natural, UNIQUE)	full_name
3f2a9c7e-6b10-4a....-b1	STU-0001	Sara Adib
9c01d4e2-11af-4d....-7e	STU-0002	Omar Reyes
a7bb30f9-2c84-4f....-02	STU-0003	Mina Lee

The id never changes and other tables point at IT. The student_code is what staff type and read; UNIQUE stops two students sharing a code. Best of both. (In YOUR practice designs you may use a plain auto-number id instead of a UUID - simpler to write by hand on paper. Same idea, same role; just smaller.)

COMPOSITE KEYS - sometimes identity takes more than one column. In a table that records "student X is a member of group Y", the pair (student, group) together is the identity - neither alone is unique. A key made of several columns is a COMPOSITE key, and it is completely normal, especially in the link tables of Part 5.

Drawn: the pair (student_id, group_id) together is the key. Look - no single column is unique, but no PAIR repeats:

student_id	group_id	joined_on	PK = (student_id, group_id)
STU-0001	G-ART	2026-09-01	STU-0001 appears twice - fine
STU-0001	G-MATH	2026-09-01	G-ART appears twice - fine
STU-0002	G-ART	2026-09-02	but (STU-0001,G-ART) only once

Trying to add (STU-0001, G-ART) again is refused by the composite key - you cannot enrol the same student in the same group twice. The key encodes a rule.

>> CONTEMPLATE: UUID vs a simple counting number for a primary key. The counter is smaller and human-readable but reveals how many rows exist and collides if you ever merge two databases. A UUID is bulky and ugly but unique across every machine on earth and leaks nothing. Which fits a small library? Which fits an app that might sync between devices? (This is a real student-panel choice.)

FOREIGN KEYS - how a relationship becomes real

A relationship in your head becomes, in tables, a FOREIGN KEY: a column in one table that holds the primary-key value of a row in another table. The Loan table has a book_id column whose value is the id of some row in Book. That column is a foreign key; it POINTS. Following it is how the database re-joins what your design split apart.

Drawn: a book carries its publisher's id. The arrow is the pointer:

PUBLISHER (the "one")

pub_id	name
PUB-1	Chatto
PUB-2	Gollancz

BOOK (the "many", holds the FK)

book_id	title	pub_id (FK)
B-1	Dune	PUB-2
B-2	Emma	PUB-1
B-3	The Hobbit	PUB-2

```
  B-1's pub_id = PUB-2, so "Dune is published by Gollancz." B-3 points at PUB-2
  too. The publisher's name is stored ONCE (in PUBLISHER) and pointed at from
  many books - exactly the "each fact in one place" rule, made real by a key.


  REFERENTIAL INTEGRITY is the promise the database enforces: a foreign key must
  point at a row that actually exists (or be empty, if the link is optional). It
  will refuse to store a loan for book #9999 if no such book exists, and it will
  stop you deleting a book that still has loans - unless you tell it what should
  happen instead. Those "what happens on delete" rules (cascade the delete,
  set the link empty, or forbid the delete) are themselves design decisions
  about what the world allows. A foreign key left pointing at a row that no
  longer exists is an ORPHAN - referential integrity exists precisely to make
  orphans impossible.

  THE "WHAT HAPPENS WHEN THE PARENT IS DELETED" RULES, as a plain sentence each.
  Say the sentence aloud and pick the one that is TRUE of your world:
  +----------------+------------------------------+--------------------------+
  | ON DELETE rule | what happens to the children | the rule as a sentence   |
  +----------------+------------------------------+--------------------------+
  | RESTRICT       | the delete is refused while  | "you may not delete a book|
  | (the default   | any child still points here | while copies of it exist" |
  |  you want most)|                              |                          |
  | CASCADE        | the children are deleted too | "deleting an order deletes|
  |                |                              | its order-lines with it"  |
  | SET NULL       | the child survives; its FK   | "if a category is deleted,|
  |                | becomes NULL (link cleared)  | its products become       |
  |                | (FK must be allowed NULL)    | uncategorised, not lost"  |
  +----------------+------------------------------+--------------------------+
  The rule as one plain sentence: a foreign key must always point at a real row
  or at nothing - never at a ghost. RESTRICT/CASCADE/SET NULL are just the three
  honest answers to "...so what do we do when the real row goes away?"

WHERE DOES THE FOREIGN KEY GO? A rule you can lean on:
  - In a ONE-TO-MANY, the key goes on the MANY side. Each book has one publisher,
    so book carries publisher_id - not the other way round. (Put it on the "one"
    side and you would need many values in one cell, which we forbade.)
  - In a ONE-TO-ONE, it can go on either side; put it where participation is
    optional, or where it reads most naturally.
  - In a MANY-TO-MANY, neither side can hold it - which forces the helper table
    of Part 5, and the foreign keys live THERE.


  The FK lives on the many side - drawn, with the crow's foot sitting exactly
  where the FK lives:
  +-----------+                                  +-------------------+
  | PUBLISHER |--||================================o<--| BOOK            |
  +-----------+                                  | ...               |
                                                 | pub_id  <- FK HERE |
                                                 +-------------------+
     the "one" side                              the "many" side holds the FK
     (no FK here)                                and wears the crow's foot
  Memorise the pairing: crow's foot = many side = where the FK column lives =
  the child pointing at its parent.

KEY IDEA: a primary key is a row's name; a foreign key is another row saying
that name to point at it. Relationships are just keys pointing at keys.

TRY IT: in the Member/Loan/Copy picture from Part 2.3, write which table holds
   which foreign key, and for each pick an ON DELETE rule and say it aloud. (Hint:
   Loan is the many side of both links. Should deleting a member delete their
```

loan history, or be refused? There is no single right answer - say your reason. That is Part 9 thinking, arriving early.)

PART 4 - READING AND DRAWING ER DIAGRAMS

A picture lets you see a whole design at once and spot what is wrong. There are a couple of notations; we use the CROW'S FOOT style, because it is the most common today and it is what your student-panel diagram uses.

THE PIECES

- Each ENTITY is a box. Inside, one line per attribute (many tools also tag the key attributes). The box is the table.
- A RELATIONSHIP is a LINE between two boxes.
- The ENDS of the line carry the cardinality - read each end as the (minimum, maximum) pair from Part 2.4.

THE THREE GLYPHS - this is the whole alphabet:

glyph	means	note
|	ONE	a single bar
o	ZERO / optional	a small circle
<	MANY	the "crow's foot" (also drawn > on a left end)

THE ONE RULE FOR READING THE MARKS. Each end of a line has TWO marks. Read them OUTSIDE-IN, toward the box:

- the FAR mark (further from the box, toward the line's middle) = the MINIMUM.
  o = optional (minimum zero) | = mandatory (minimum one)
- the NEAR mark (right against the box) = the MAXIMUM.
  | = one (maximum one) < = many (maximum many)

So "near tells you one-or-many; far tells you optional-or-mandatory."

THE FOUR END-COMBINATIONS. Here written for an entity on the RIGHT of the line (so the order left-to-right is far-mark then near-mark, i.e. MIN then MAX):

right-end mark	reads as	meaning (min, max)
---o| BOX	zero or one	optional, at most one (0,1)
---|| BOX	exactly one	mandatory, exactly one (1,1)
---o< BOX	zero or many	optional, many (0,many)
---|< BOX	one or many	mandatory, at least one (1,many)

THE MIRROR. The marks flip left-to-right when the entity is on the LEFT of the line - the crow's foot points the other way and the pair reverses, but the MEANING is identical. "o< on a right end" is the same as ">o on a left end":

left-end mark	right-end mirror	both mean
BOX |o---	---o| BOX	zero or one
BOX ||---	---|| BOX	exactly one
BOX >o---	---o< BOX	zero or many
BOX >|---	---|< BOX	one or many

Do not let the flip worry you: pick the box, find its two marks, read near=max and far=min. It says the same thing from either side.

READING A LINE
  You read a crow's-foot line from each side toward the other. For
     Publisher ||--o< Book
  read left-to-right: "one publisher has zero-or-many books," and right-to-left:
  "each book has exactly one publisher." Both sentences must be TRUE of the real
  world - saying them aloud is the best test of your own diagram.

THE COMMON SHAPES, DRAWN (compact inline AND as boxes - learn both forms):
  1:1   Country ||--|| Capital   one and only one each way
  1:N   Publisher ||--o< Book   one publisher, many books
  M:N   Student >o--o< Course   many each way - needs a helper (Part 5)

  optional 1:N worth staring at - a member may have NO loans (o near Loan's
  "many" is really about the member end), yet every loan MUST have exactly one
  copy. Drawn as boxes:
  +-----------+   +-----------+
  | MEMBER |--||============================o<--| LOAN |
  +-----------+   +-----------+
     exactly one member   zero or many loans
     -> one MEMBER has zero or many LOANs   (a brand-new member has zero)
     <- each LOAN belongs to exactly one MEMBER (never a loan with no member)

  +-----------+   +-----------+
  | LOAN |--||============================||--| COPY |
  +-----------+   +-----------+
     -> each LOAN is of exactly one COPY
     <- each COPY is on exactly one LOAN AT A TIME (over time, many - see note)
  The little circle vs bar near the box is where "is this allowed to be empty?"
  lives. Change one circle to a bar and you have changed what the world permits.

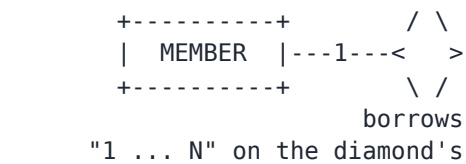
A TINY WORKED DIAGRAM (in text)
  Author ||--o< Book   one author writes many books; each book, one author
  Book ||--o< Copy   a title has many physical copies; each copy, one title
  Read those four sentences aloud. If any is false for a real library (maybe a
  book has several authors?), the diagram is wrong and must change - better to
  find it now, on paper, than after building. (A book with several authors is
  itself a many-to-many - and you now know what that needs. Part 5.)

A WHOLE SMALL MODEL AT A GLANCE (preview of Part 8's library; full tables there).
See how the boxes and marks let you take in an entire design in one look:

  +----------+   +----------+   +----------+
  | AUTHOR |--||====o<--| BOOK |--||====o<--| COPY |
  +----------+   +----------+   +----------+
    one author   one book   each copy belongs to one
    many books   many copies   book; has many loans over
                                                     time... |
                                                              o<
                                                              |
  +----------+   +--------------+
  | MEMBER |--||====================================o<--| LOAN |
  +----------+   +--------------+
    one member has zero-or-many loans; each loan has exactly one member AND
    exactly one copy. LOAN sits at the "many" end of two links - it is a bridge.

ANOTHER NOTATION YOU'LL SEE (Chen)
  The older Chen notation draws relationships as DIAMONDS between the boxes, with
  numbers (1, N, M) on the lines, and attributes as ovals. It is common in

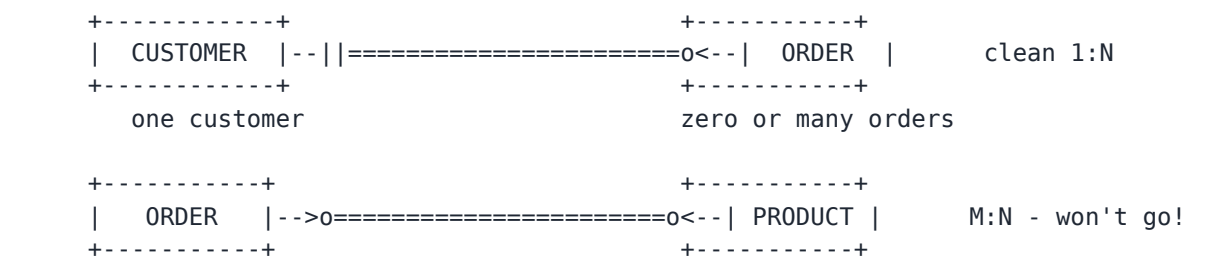
textbooks. You don't need to draw it, only to recognise it: a diamond labelled "borrows" between Member and Copy is the same idea as a crow's-foot line.



"1 ... N" on the diamond's arms = one member borrows N copies. Same meaning as Member ||--o< Copy in crow's foot. Different picture, identical idea.

TRY IT: on your blank sheet, draw Customer, Order, and Product as three boxes. A customer places many orders; each order belongs to one customer. An order contains many products, and a product appears on many orders. Draw the lines, put the right symbols on the ends, and say all four sentences aloud. You will find one line you cannot draw cleanly - hold that feeling; Part 5 is its cure.

WHAT YOU PROBABLY DREW (and the line that fought back):



many each way: one order has many products, one product on many orders
The Customer-Order line is a tidy 1:N. The Order-Product line is MANY-TO-MANY, and there is nowhere to hang a foreign key - that is the feeling to hold. Part 5.1 names the fix (it is the order LINE), and you will have met it twice before you need it.

PART 5 - THE PATTERNS BEGINNERS TRIP ON

Everything so far handles the easy links. A handful of situations need a specific move each. Learn these five and very little in real design will surprise you. Here they are on one card, with the single move each one needs:

#	situation	the one move
1	many-to-many	a BRIDGE table (one row per pairing)
2	relationship w/ own facts	that bridge/table HOLDS the facts
3	one-to-one	usually MERGE; split only on purpose
4	self-reference (a tree)	a parent_id FK into the SAME table
5	ternary (three-way fact)	a table in the MIDDLE with three FKs

Notice how often the move is the same shape: a table in the middle holding foreign keys. Learn that one move and most "hard" designs become routine.

5.1 MANY-TO-MANY NEEDS A BRIDGE TABLE (the big one)

A student takes many courses; a course holds many students. Try to store this with a foreign key and you fail both ways: you cannot list many course-ids in a student's single cell, nor many student-ids in a course's. A table cell holds one value - so a direct many-to-many is impossible.

THE IMPOSSIBLE ATTEMPT, drawn - a comma-list in a cell (breaks 1NF, Part 6):

```
  | student_id | name        | courses (DO NOT DO THIS)   |
  +------------+-------------+----------------------------+
  | STU-0001   | Sara Adib   | ART101, MATH210, HIST105   |  <- many in one cell
  | STU-0002   | Omar Reyes  | MATH210                    |
  +------------+-------------+----------------------------+
  You cannot ask "who is in MATH210?" without picking strings apart; you cannot
  store a grade per course; adding a course means editing text. Dead end.


The cure: invent a THIRD table that holds one row per pairing. Call it Enrolment.
Each row has one student_id and one course_id - two foreign keys - and together
they identify the row (a composite key, Part 3). Now:
     Student ||--o< Enrolment >o--|| Course
The impossible M:N has become two easy 1:N links through a table in the middle.
This helper is called a BRIDGE, JUNCTION, LINK, or ASSOCIATIVE table - same thing.

THE CURE, drawn as boxes - two tidy 1:N links meeting in the middle:
  +-----------+                +---------------+                +----------+
  |  STUDENT  |--||=========o<--| ENROLMENT     |--o<=========||--| COURSE   |
  +-----------+                | student_id FK |                +----------+
  one student has              | course_id  FK |              one course has
  many enrolments              | grade         |              many enrolments
                               +---------------+
     ->  one STUDENT has many ENROLMENTs; each ENROLMENT is for one STUDENT
     <-  one COURSE has many ENROLMENTs; each ENROLMENT is for one COURSE

THE CURE, drawn as rows that connect (keep the ids consistent and follow them):
  STUDENT                        COURSE
  +----------+-----------+       +-----------+------------------+
  | stud_id  | name      |       | course_id | title            |
  +----------+-----------+       +-----------+------------------+
  | STU-0001 | Sara Adib |       | ART101    | Drawing          |
  | STU-0002 | Omar Reyes|       | MATH210   | Calculus         |
  +----------+-----------+       | HIST105   | Modern History   |
                                 +-----------+------------------+
  ENROLMENT  (the bridge; PK = (stud_id, course_id))
  +----------+-----------+------------+--------+
  | stud_id  | course_id | enrolled_on| grade  |
  +----------+-----------+------------+--------+
  | STU-0001 | ART101    | 2026-09-01 | A      |   Sara is in Drawing...
  | STU-0001 | MATH210   | 2026-09-01 | B      |   ...and Calculus...
  | STU-0001 | HIST105   | 2026-09-02 | (null) |
  | STU-0002 | MATH210   | 2026-09-01 | A      |   Omar is in Calculus
  +----------+-----------+------------+--------+
  Now "who is in MATH210?" = enrolment rows with course_id=MATH210 -> Sara,Omar.
  "Sara's courses?" = rows with stud_id=STU-0001 -> Drawing, Calculus, History.


THE BONUS: this bridge is the natural home for facts about the PAIRING itself -
the enrolment date, the final grade, whether it was dropped. Remember from Part
2.3 that a relationship can have its own attributes; the bridge is WHERE they
live. When a relationship earns its own facts, it has quietly become an entity.
Above, enrolled_on and grade belong to the student-and-course PAIR - nowhere else.

COMPOSITE PK vs OWN SURROGATE id - a choice you must make for every bridge.
  Two honest options; pick one and WRITE DOWN which:
  +---------------------------+-------------------------------------------+
  | option                    | consequence                               |
  +---------------------------+-------------------------------------------+
  | PK = (stud_id, course_id) | the pair IS the identity; no two rows can |
  | (composite, no extra id)  | repeat a pairing - the rule is free       |
```

```
  | PK = enrolment_id          | a tidy single id to point at from elsewhere, |
  | (own surrogate) + UNIQUE   | BUT you MUST add UNIQUE(stud_id, course_id)  |
  | (stud_id, course_id)       | or duplicates sneak back in                  |
  +----------------------------+----------------------------------------------+
  The rule to say every time: if a bridge gets its own surrogate id, decide
  whether the natural combination should carry a UNIQUE constraint - and here it
  must, because "the same student in the same course twice" is nonsense. (This
  is exactly how student-panel's scores table behaves: it bridges a student and
  a parameter for a period, and the combination is kept unique.)

>> CONTEMPLATE: your Customer-Order-Product sketch from Part 4. Order-to-Product
   is many-to-many. What is the bridge? (It is the order LINE: one row per "this
   order, this product, this quantity, this price".) Notice the bridge holds
   facts - quantity, price - that belong to neither the order nor the product
   alone. That is the tell that it deserved to exist.


5.2  ONE-TO-ONE: usually a question, sometimes a choice
-------------------------------------------------------
A true 1:1 is uncommon. When you find one, ask first: should these be one
entity? A Person and their Passport-details might just be more columns on Person.
But 1:1 splits are made ON PURPOSE when: the extra data is rarely present (keep
it out of the main table so most rows stay lean), or it is sensitive and kept
separate for access control, or it is huge (a profile photo) and you don't want
to drag it along every time you read the main row. So 1:1 = "merge, unless you
have a reason to split" - and know your reason.

MERGED (the usual answer) - one table, passport columns may be NULL:
  +----------+-------------+---------------+-----------------+
  | person_id| name        | passport_no   | passport_expiry |
  +----------+-------------+---------------+-----------------+
  | P-1      | Sara Adib   | 981222345     | 2030-05-01      |
  | P-2      | Omar Reyes  | (null)        | (null)          |   no passport
  +----------+-------------+---------------+-----------------+

SPLIT ON PURPOSE - sensitive/rare data in its own table; the FK goes on the
never-empty side (passport) with UNIQUE + NOT NULL, so it stays truly 1:1:
  PERSON                            PASSPORT (zero-or-one per person)
  +----------+-------------+         +-----------+-----------+-------------+
  | person_id| name        |         | person_id | number    | expiry      |
  +----------+-------------+         | (FK,UNIQUE|           |             |
  | P-1      | Sara Adib   |<--------|  NOT NULL)|           |             |
  | P-2      | Omar Reyes  |         +-----------+-----------+-------------+
  +----------+-------------+         | P-1       | 981222345 | 2030-05-01  |
                                     +-----------+-----------+-------------+
  person_id in PASSPORT is UNIQUE (so one person cannot have two passport rows)
  and NOT NULL (so a passport row cannot float free of a person). Those two
  constraints are what MAKE it one-to-one rather than one-to-many. P-2 simply
  has no row here - that is the "zero" in "zero or one."

  Drawn as a line:
  +----------+                              +------------+
  |  PERSON  |--|o============================||--| PASSPORT |
  +----------+                              +------------+
     ->  one PERSON has zero or one PASSPORT
     <-  each PASSPORT belongs to exactly one PERSON


5.3  SELF-REFERENCING RELATIONSHIPS: trees and hierarchies
----------------------------------------------------------
Sometimes an entity relates to OTHERS OF ITS OWN KIND. An employee has a manager
```

who is another employee. A category has a parent category. A comment replies to another comment. You model this with a foreign key that points back into the SAME table: employee.manager_id points at another employee.id.

Drawn as rows - manager_id points at another row's id in the SAME table; the boss has manager_id = NULL (the root of the tree):

```
  +--------+---------------+--------------+
  | emp_id | name          | manager_id   |
  +--------+---------------+--------------+
  | E-1    | Dana (CEO)    | (null)       |   <- root: reports to no one
  | E-2    | Raj           | E-1          |   Raj reports to Dana
  | E-3    | Mina          | E-1          |   Mina reports to Dana
  | E-4    | Sol           | E-2          |   Sol reports to Raj
  | E-5    | Yara          | E-2          |   Yara reports to Raj
  +--------+---------------+--------------+
```

The same rows, drawn as the TREE they encode:

```
          E-1 Dana (CEO)            manager_id = NULL
          /          \
       E-2 Raj      E-3 Mina        manager_id = E-1
       /     \
    E-4 Sol  E-5 Yara               manager_id = E-2
```

Drawn as a line that loops back to the same box:

```
  +----------------+
  |   EMPLOYEE     |----+
  |  emp_id        |    |  "is managed by"
  |  manager_id FK |    |   o| (zero or one manager: the root has none)
  +----------------+    |   |<  on the other arm (a manager has one-or-many
          ^             |        reports) - same box, both ends
          +-------------+
```

This builds a TREE: a top row whose parent is empty (NULL - the root), rows beneath it, rows beneath those, to any depth. It is how you store "folders inside folders" shapes with no fixed number of levels. student-panel uses this twice - student groups can nest inside groups, and parameters inside parameters - so it is worth getting comfortable with now. (student-panel also PREVENTS CYCLES: a group may not end up being its own ancestor - a rule the self-reference alone does not enforce, so the app checks it.)

>> CONTEMPLATE: why can't you just make columns parent, grandparent, great-grandparent? Because you would have to guess the maximum depth forever, and most rows would leave them empty. A single self-pointing parent_id handles ANY depth with one column. Recursion beats a fixed guess. (Asking "give me the whole branch under this node" then needs a special looping query - a real cost of this elegant trick, and a fair thing to weigh.)

>> PITFALL: forgetting the root. SOMETHING has to sit at the top with no parent. If every row insists on a non-NULL parent_id, you can never insert the first row (nothing exists yet to point at) - an insert anomaly born of a too-strict rule. Let the root's parent_id be NULL, and give that NULL the plain meaning "this is a top-level node."

5.4 WEAK / DEPENDENT ENTITIES

Some things cannot exist on their own. An order-line has no meaning without its order; a room has no meaning without its building. These are WEAK entities: they depend on a "parent" (the strong entity) for their very existence, and often for part of their identity too (order-line #3 only makes sense as "line 3 OF order #88"). In practice you model a weak entity as its own table with a mandatory

foreign key to its parent, and usually an ON-DELETE rule that removes the children when the parent goes (the lines die with their order). The design question a weak entity answers is "can this ever stand alone?" - and the answer shapes both its key and its delete behaviour.

Drawn - an order and its lines (note: the table is "orders", never "order" - "order" is a reserved word in SQL; see Part 9):

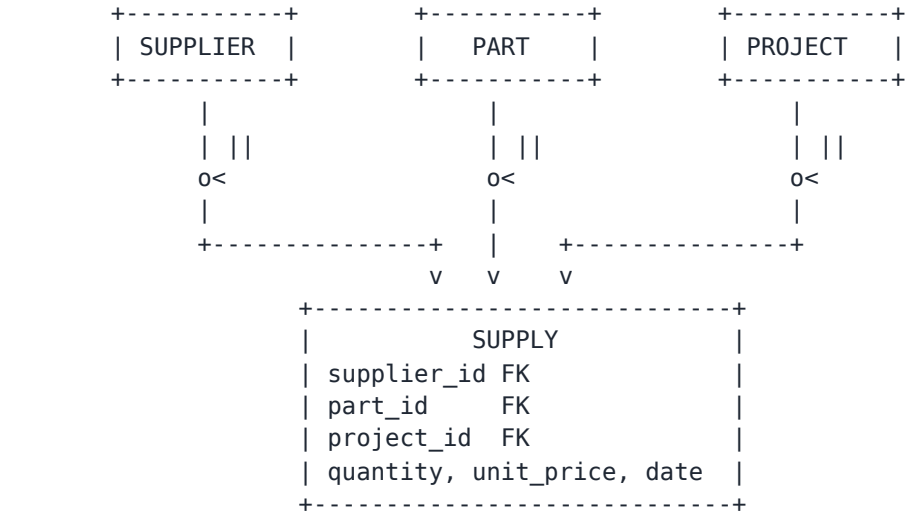
```
  ORDERS (strong)                ORDER_LINE (weak; dies with its order)
  +----------+------------+       +----------+---------+----------+-----+
  | order_id | placed_on  |       | order_id | line_no | product  | qty |
  +----------+------------+       | (FK, PK  |         |          |     |
  | O-88     | 2026-09-30 |<------|  part)   |         |          |     |
  | O-89     | 2026-10-01 |<--+   +----------+---------+----------+-----+
  +----------+------------+   |   | O-88     | 1       | Pencils  | 3   |
                              |   | O-88     | 2       | Eraser   | 1   |
                              +---| O-89     | 1       | Notebook | 2   |
                                  +----------+---------+----------+-----+
```

PK of ORDER_LINE = (order_id, line_no): "line 1 of O-88" - line_no alone means nothing. order_id is mandatory (NOT NULL): a line with no order is impossible. ON DELETE CASCADE: delete O-88 and its two lines go with it (they cannot outlive it). That is the weak entity's whole character in one picture.

5.5 TERNARY RELATIONSHIPS: prefer an entity in the middle

"A supplier supplies a part to a project" links three entities in one fact. You CAN draw a three-way relationship, but it is hard to read and harder to add facts to. The usual move is the same as for many-to-many: create an entity in the middle - a "Supply" row with three foreign keys (supplier, part, project) plus its own facts (quantity, price, date). Almost every messy multi-way relationship becomes clean once you let the relationship itself become a table.

Drawn - three parents point INTO one middle table; the fact lives in the middle:



As rows (the three FKs plus the pairing's own facts):

supplier_id	part_id	project_id	qty	unit_price	supplied_on
SUP-1	P-BOLT	PRJ-BRIDGE	500	0.12	2026-09-10
SUP-1	P-BOLT	PRJ-TOWER	200	0.12	2026-09-11
SUP-2	P-NUT	PRJ-BRIDGE	500	0.08	2026-09-12

If SUPPLY gets its own surrogate supply_id, decide whether the natural TRIPLE (supplier_id, part_id, project_id) should be UNIQUE. Here: can the same supplier supply the same part to the same project on two dates? If yes, the

```
    triple is NOT unique by itself and the date is part of what distinguishes rows;
    if no, add UNIQUE on the triple. Say which, and why - that is the design.

  KEY IDEA: whenever a link is many-to-many, self-referencing, or carries its own
  facts, the answer is almost always the same shape - a table in the middle whose
  rows are the pairings (or triplings), holding foreign keys and any facts of their
  own. Learn that one move and most "hard" designs become routine.


--------------------------------------------------------------------------------
  PART 6 - NORMALISATION: PUTTING EACH FACT IN ONE PLACE
--------------------------------------------------------------------------------

Normalisation sounds intimidating and is actually one idea repeated: organise
your tables so that each fact is stored exactly once. Everything below is that
idea, made progressively stricter.

  THE DISEASE: redundancy and its "anomalies"
    Picture one wide table, one row per loan, repeating the member's name, phone,
    and address on every single loan they ever make. Here it is with enough rows
    to feel the trouble:

    +---------+-------------+--------------+------------+------------+
    | loan_id | member_name | member_phone | book_title | due_date   |
    +---------+-------------+--------------+------------+------------+
    | 1       | Sara Adib   | 555-1020     | Dune       | 2026-10-14 |
    | 2       | Sara Adib   | 555-1020     | Emma       | 2026-10-14 |
    | 3       | Omar Reyes  | 555-7788     | Dune       | 2026-10-20 |
    | 4       | Sara Adib   | 555-1020     | The Hobbit | 2026-10-28 |
    +---------+-------------+--------------+------------+------------+
    Sara's name and phone are written three times. That triple-storage is the
    disease; the three "anomalies" below are its symptoms.


    Three illnesses follow, and they are worth naming because you will feel them.
    Each is drawn as the exact bad moment it causes:

    UPDATE ANOMALY - Sara changes her phone; you fix some rows and miss one:
    +---------+-------------+--------------+------------+
    | loan_id | member_name | member_phone | book_title |
    +---------+-------------+--------------+------------+
    | 1       | Sara Adib   | 555-2000 (new)| Dune      |
    | 2       | Sara Adib   | 555-2000 (new)| Emma      |
    | 4       | Sara Adib   | 555-1020 (OLD!)| The Hobbit|  <- missed
    +---------+-------------+--------------+------------+
    The table now claims Sara has two phone numbers and cannot say which is true.

    INSERT ANOMALY - a new member, Mina, who has not borrowed anything yet:
    +---------+-------------+--------------+------------+
    | loan_id | member_name | member_phone | book_title |
    +---------+-------------+--------------+------------+
    | ???     | Mina Lee    | 555-3030     | (null?)    |  <- no loan to attach to
    +---------+-------------+--------------+------------+
    There is nowhere to record Mina. The table is ABOUT loans, so a member with no
    loan has no home. Her existence wrongly depends on her having borrowed a book.

    DELETE ANOMALY - Omar returns his only book and you delete that one loan:
    +---------+-------------+--------------+------------+
    | loan_id | member_name | member_phone | book_title |
    +---------+-------------+--------------+------------+
    | 3       | Omar Reyes  | 555-7788     | Dune       |  <- delete this row...
```

```
  +---------+------------+-------------+------------+
  ...and Omar vanishes entirely - his name and phone existed ONLY on that row.
  You meant to erase a loan; you erased a person.

  All three come from ONE mistake: two different KINDS of fact (facts about the
  member, facts about the loan) crammed into one table. The cure is to split
  them so each kind of fact lives in its own table, linked by a key.


THE CURE, IN STEPS (the "normal forms")
  Each form fixes one kind of muddle. You mostly aim for the third.

  FIRST NORMAL FORM (1NF) - atomic values, no repeating groups.
    Every cell holds ONE value, and no row has "columns" like phone1, phone2,
    phone3. If a thing has many values, it belongs in its own rows/table (the
    multi-valued attribute of Part 2.2, and the bridge of Part 5). Tidy cells.

    NOT 1NF (repeating group of phone columns; most cells empty, max guessed):
    +----------+-------------+----------+----------+----------+
    | memb_id  | name        | phone1   | phone2   | phone3   |
    +----------+-------------+----------+----------+----------+
    | M-1001   | Sara Adib   | 555-1020 | 555-3344 | (null)   |
    | M-1002   | Omar Reyes  | 555-7788 | (null)   | (null)   |
    +----------+-------------+----------+----------+----------+
    1NF (one value per cell; many phones become many ROWS in their own table):
    MEMBER                         MEMBER_PHONE
    +----------+-------------+     +----------+----------+
    | memb_id  | name        |     | memb_id  | number   |
    +----------+-------------+     +----------+----------+
    | M-1001   | Sara Adib   |     | M-1001   | 555-1020 |
    | M-1002   | Omar Reyes  |     | M-1001   | 555-3344 |
    +----------+-------------+     | M-1002   | 555-7788 |
                                   +----------+----------+

  SECOND NORMAL FORM (2NF) - no partial dependency on part of a composite key.
    Only bites when your key is composite. In an Enrolment keyed by (student,
    course), the student's NAME depends on the student alone - only part of the
    key - so it does not belong here; it belongs in Student. Every non-key fact
    must depend on the WHOLE key, not half of it.


    NOT 2NF (key is (stud_id, course_id); but student_name depends on stud_id
    ALONE, and course_title on course_id ALONE - partial dependencies):
    +----------+-----------+--------------+---------------+--------+
    | stud_id  | course_id | student_name | course_title  | grade  |
    +----------+-----------+--------------+---------------+--------+
    | STU-0001 | ART101    | Sara Adib    | Drawing       | A      |
    | STU-0001 | MATH210   | Sara Adib    | Calculus      | B      |
    | STU-0002 | MATH210   | Omar Reyes   | Calculus      | A      |
    +----------+-----------+--------------+---------------+--------+
     "Sara Adib" and "Calculus" repeat - and could disagree between rows.
    2NF (each partial fact moves to the table of the key-part it depends on;
    only grade - which needs BOTH parts - stays on the bridge):
    STUDENT                 COURSE                  ENROLMENT
    +----------+---------+  +-----------+---------+ +----------+-----------+-----+
    | stud_id  | name    |  | course_id | title   | | stud_id  | course_id |grade|
    +----------+---------+  +-----------+---------+ +----------+-----------+-----+
    | STU-0001 | Sara    |  | ART101    | Drawing | | STU-0001 | ART101    | A   |
    | STU-0002 | Omar    |  | MATH210   | Calculus| | STU-0001 | MATH210   | B   |
    +----------+---------+  +-----------+---------+ | STU-0002 | MATH210   | A   |
                                                    +----------+-----------+-----+
```

THIRD NORMAL FORM (3NF) - no transitive dependency.
A non-key column must depend on the key, not on ANOTHER non-key column. If a Book table stores publisher_id AND publisher_address, the address really depends on the publisher, not on the book - so it belongs in a Publisher table. Facts about a publisher live with the publisher.

NOT 3NF (book_id is the key, but publisher_name/address depend on pub_id, another non-key column - a transitive dependency book -> pub -> address):

book_id	title	pub_id	publisher_name	publisher_address
B-1	Dune	PUB-2	Gollancz	50 Victoria, London
B-2	Emma	PUB-1	Chatto	20 Vauxhall, London
B-3	The Hobbit	PUB-2	Gollancz	50 Victoria, London

Gollancz's address repeats on B-1 and B-3 - change the address once and the rows disagree. 3NF (publisher facts move to a PUBLISHER table; the book just keeps the pointer):

BOOK

book_id	title	pub_id
B-1	Dune	PUB-2
B-2	Emma	PUB-1
B-3	The Hobbit	PUB-2

PUBLISHER

pub_id	name	address
PUB-1	Chatto	20 Vauxhall, London
PUB-2	Gollancz	50 Victoria, London

one row per publisher; address stored once

(There is a stricter cousin, BCNF, and higher forms for rare cases. For almost all real work, "clean to 3NF" is the target and the rest is specialist.)

THE MNEMONIC: every non-key fact must depend on "the key, the whole key, and nothing but the key." That single line is 1NF-2NF-3NF in a sentence.

form	the phrase	what it forbids
1NF	(atomic cells)	many values in a cell; phone1,2,3
2NF	"...the WHOLE key..."	a fact depending on PART of the key
3NF	"...and NOTHING BUT the key."	a fact depending on another non-key

WHEN TO STOP - and even step back
Normalisation splits data into more, smaller tables. That is good for correctness but means more links to re-join when you ask questions, which can be slower. Sometimes, deliberately and knowingly, designers DENORMALISE - keep a little redundancy - for speed on read-heavy systems. Do it consciously, never by accident, and never before you have a measured reason. Correct first; fast later, only if needed.

DENORMALISE vs a FROZEN FACT - do not confuse them. Both put a "copyable" value in a row, but for opposite reasons:

	denormalised copy	frozen fact
why it is there	speed; avoid a join	it was TRUE at a moment and must not change later
may it drift from the source?	no - must be kept in sync with the source	it is SUPPOSED to differ from today's value
example	copy publisher_name onto book for fast lists	unit_price_at_sale on an order line (Part 9)

```
  +-----------------+--------------------------+--------------------------+
  A frozen fact is not a normalisation sin - it is a DIFFERENT fact that happens
  to look like today's value. Part 9 gives it its due.

>> CONTEMPLATE: go back to the loan table above and split it into clean tables.
   How many do you get, and what is the key and the foreign key of each? (You
   should reach Member, Book, and Loan - the bridge - with Loan carrying member_id
   and book_id. You just normalised a design by yourself.)


THE SICK TABLE, CURED - the wide loan sheet from the top of this Part, split so
each fact sits once. Follow the ids across the three tables:
  MEMBER                              BOOK
  +----------+-------------+--------+ +--------+------------+
  | memb_id  | name        | phone  | | book_id| title      |
  +----------+-------------+--------+ +--------+------------+
  | M-1001   | Sara Adib   |555-1020| | B-1    | Dune       |
  | M-1002   | Omar Reyes  |555-7788| | B-2    | Emma       |
  | M-1003   | Mina Lee    |555-3030| | B-3    | The Hobbit |
  +----------+-------------+--------+ +--------+------------+
  LOAN  (the bridge; member_id FK -> MEMBER, book_id FK -> BOOK)
  +---------+----------+---------+------------+
  | loan_id | memb_id  | book_id | due_date   |
  +---------+----------+---------+------------+
  | 1       | M-1001   | B-1     | 2026-10-14 |   Sara / Dune
  | 2       | M-1001   | B-2     | 2026-10-14 |   Sara / Emma
  | 3       | M-1002   | B-1     | 2026-10-20 |   Omar / Dune
  | 4       | M-1001   | B-3     | 2026-10-28 |   Sara / The Hobbit
  +---------+----------+---------+------------+
  Now watch the three anomalies die: Sara's phone is in ONE place (change it
  once); Mina (M-1003) exists with NO loans at all - no problem; delete loan 3
  and Omar still exists in MEMBER. One fact, one place, every symptom gone.


--------------------------------------------------------------------------------
  PART 7 - THE METHOD: A REPEATABLE RECIPE
--------------------------------------------------------------------------------

Now that you own the pieces, here is the order to use them. Follow it top to
bottom the first few times; later it becomes instinct. Notice it moves from ideas
(steps 1-4) to tables (steps 8-9), exactly the level-1-to-level-2 descent of Part 1.

To make the steps concrete, we DO each one on a small running example as we go: a
CLINIC that books patients with doctors and records what gets prescribed. (The
full library example is Part 8; here the clinic shows the METHOD in motion.)

  STEP 1  STATE THE PURPOSE. Write, in sentences, what the system is for, who uses
          it, and - crucially - the QUESTIONS it must answer and ACTIONS it must
          support (Part 1). This list is your ruler for everything after.

     DOING IT (clinic):
     "A clinic books appointments between patients and doctors, and records the
      drugs prescribed at a visit."
     +----+-----------------------------------------------+
     | Q1 | which appointments does Dr Khan have today?   |
     | Q2 | what was prescribed to patient P at visit V?  |
     | Q3 | which patients has doctor D ever seen?        |
     | A1 | book an appointment; A2 record a prescription|
     +----+-----------------------------------------------+

  STEP 2  FIND THE ENTITIES. Underline the NOUNS in your sentences. Group synonyms,
```

drop the ones you keep no facts about. What's left are your candidate entity types (tables-to-be).

DOING IT (clinic): nouns -> PATIENT, DOCTOR, APPOINTMENT (a visit; it will carry its own facts - a time), DRUG. "clinic" itself is just the whole system, not a table. Four candidate entities.

STEP 3 FIND THE ATTRIBUTES. For each entity, list what you know about it. Mark which are composite (split them), which are multi-valued (they will become their own table), and which are derived (don't store them).

DOING IT (clinic):

entity	attributes (composite split; derived dropped)
PATIENT	patient_id, given_name, family_name (split), birthdate
DOCTOR	doctor_id, name, speciality
APPOINTMENT	appt_id, patient_id, doctor_id, starts_at
DRUG	drug_id, name, strength

(A patient's AGE is derived from birthdate - not stored.)

STEP 4 FIND THE RELATIONSHIPS. Underline the VERBS joining nouns. For each, ask cardinality in BOTH directions and its participation (min/max, Part 2.4). Say each relationship aloud as two sentences and check both are true.

DOING IT (clinic):

relationship	both directions	type
PATIENT - APPOINTMENT	patient has many appts; appt 1 pat.	1:N
DOCTOR - APPOINTMENT	doctor has many appts; appt 1 doc.	1:N
APPOINTMENT - DRUG	a visit prescribes many drugs; a drug given at many visits	M:N

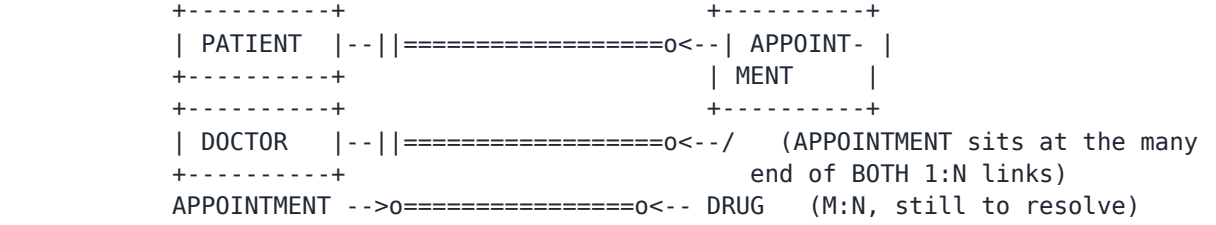
The M:N (appointment-drug) is a flag for Step 7.

STEP 5 CHOOSE KEYS. Give each entity a primary key (usually a surrogate; keep natural identifiers as unique columns). Note where composite keys arise.

DOING IT (clinic): surrogate ids patient_id, doctor_id, appt_id, drug_id. The bridge from Step 7 (prescription) will key on (appt_id, drug_id). Keep any natural code (e.g. an NHS number on patient) as a UNIQUE column, not the PK.

STEP 6 DRAW IT. Put the entities in boxes, draw the relationship lines, mark the crow's-foot ends. Now you can SEE the whole model and critique it.

DOING IT (clinic):



STEP 7 RESOLVE THE HARD PATTERNS. Turn every many-to-many into a bridge table; handle self-references, weak entities, and ternaries (Part 5). Move each relationship's own facts onto its bridge.

DOING IT (clinic): the APPOINTMENT-DRUG many-to-many becomes a PRESCRIPTION bridge, holding the pairing's own facts (dose, instructions):

```
  APPOINTMENT ||--o< PRESCRIPTION >o--|| DRUG
+----------+---------+--------------+---------------------+
| appt_id  | drug_id | dose_mg      | instructions        |
+----------+---------+--------------+---------------------+
| AP-1     | D-AMOX  | 500          | 3x daily, 7 days    |
| AP-1     | D-IBU   | 200          | as needed           |
| AP-2     | D-AMOX  | 250          | 2x daily, 5 days    |
+----------+---------+--------------+---------------------+
```

PK = (appt_id, drug_id); or a surrogate prescription_id WITH UNIQUE(appt_id, drug_id) if the same drug is never listed twice on one visit.

STEP 8 NORMALISE. Check each table to 3NF: "the key, the whole key, and nothing but the key" (Part 6). Split out any fact that is in the wrong place.

DOING IT (clinic): is the patient's name on APPOINTMENT? No - it is on PATIENT (it depends on the patient, not the visit). Is the drug's name on PRESCRIPTION? No - it is on DRUG. dose_mg needs BOTH appt and drug, so it rightly stays on the bridge. Clean to 3NF.

STEP 9 MAP TO TABLES. Apply the mapping rules below to write the final table list: names, columns, types, primary keys, foreign keys, and delete rules.

DOING IT (clinic) - the final list:

```
  patient(patient_id PK, given_name, family_name, birthdate)
  doctor(doctor_id PK, name, speciality)
  appointment(appt_id PK, patient_id FK->patient, doctor_id FK->doctor,
              starts_at)
  drug(drug_id PK, name, strength)
  prescription(appt_id FK->appointment, drug_id FK->drug, dose_mg,
               instructions, PK=(appt_id, drug_id))
```

ON DELETE: deleting an appointment CASCADEs to its prescriptions (they cannot outlive the visit); deleting a doctor is RESTRICTed while appointments exist.

STEP 10 REVIEW AGAINST STEP 1. Walk every question and action from your purpose list through the design. Can each be answered? Iterate. First drafts are never final - good designs are rewritten, not born.

DOING IT (clinic): Q1 "Dr Khan's appts today" = appointment rows with his doctor_id and today's date - yes. Q2 "prescribed to P at visit V" = prescription rows for that appt_id - yes. Q3 "patients doctor D has seen" = distinct patient_id over D's appointments - yes. All answerable; design holds.

THE MAPPING RULES (ER model -> tables), collected in one place

- Each ENTITY becomes a TABLE; each of its atomic attributes becomes a COLUMN.
- Each entity's PRIMARY KEY becomes the table's primary key.
- A ONE-TO-MANY relationship becomes a FOREIGN KEY column on the MANY side, pointing at the one side's primary key.
- A ONE-TO-ONE becomes a foreign key on either side (put it on the optional side), often with a UNIQUE rule so it stays one-to-one.
- A MANY-TO-MANY becomes a NEW bridge table whose primary key is the pair of foreign keys (plus any facts about the pairing).
- A MULTI-VALUED attribute becomes its own table, one row per value, linked by a foreign key back to the owner.
- A SELF-REFERENCE becomes a foreign key column pointing back into the same table (parent_id), NULL at the root.
- For each foreign key, decide the ON-DELETE rule: cascade (children die with the parent), set-null (link clears, child survives), or restrict (forbid

deleting while children exist). This encodes what the world allows.

The same rules as a lookup table - "when I see THIS in the model, I write THAT in the tables":

in the ER model	in the tables
entity	a table
atomic attribute	a column
primary key	the table's PRIMARY KEY
1:N relationship	FK on the many side -> one side's PK
1:1 relationship	FK on one side + UNIQUE
M:N relationship	a new bridge table (two FKs = its PK)
multi-valued attribute	its own table, one row per value
self-reference (tree)	parent_id FK into the same table, NULL at the root
each FK	pick ON DELETE: cascade/set-null/restrict

KEY IDEA: the recipe never changes - purpose, entities, attributes, relationships, keys, draw, resolve patterns, normalise, map, review. Memorise the ten steps and you can design anything, because you will always know what to do next.

PART 8 - A COMPLETE WORKED EXAMPLE: A SMALL COMMUNITY LIBRARY

Let us run the whole recipe once, start to finish, on something small. Read it with your own sheet and try each step before reading my version. This is the example to come back to: it is drawn ALL the way down to sample rows and answered questions, so you can see a finished design actually working.

STEP 1 - PURPOSE. A small library lends books to members. It must answer: "which books does this member have out now?", "is any copy of this title available?", "who is overdue?", "how many times has this title been borrowed?" Actions: register a member, lend a copy, take a return.

STEP 2 - ENTITIES (nouns underlined in the purpose): MEMBER, BOOK (the title/work), COPY (a physical volume - a title can have several), and - foreseeing a verb with its own facts - LOAN. Four entities.

STEP 3 - ATTRIBUTES.

```
MEMBER: id, full name (composite - store given/family), email (unique),
        joined_on. (Age? derived from nothing stored - skip; we don't need it.)
BOOK:   id, title, author, isbn (natural, unique). (One author, to stay small;
        note the simplification - a real library would make Author an entity.)
COPY:   id, the book it is a copy of (link), shelf code, acquired_on.
LOAN:   id, which copy, which member, borrowed_on, due_on, returned_on (NULL
        until returned - its emptiness MEANS "still out").
```

STEP 4 - RELATIONSHIPS and their cardinality (say each aloud, both ways):

```
BOOK ||--o< COPY      one title has many copies; each copy is of one title.
MEMBER ||--o< LOAN     a member may have many loans (and zero); each loan, one
                       member.
COPY ||--o< LOAN       a copy has many loans over time; each loan, one copy.
```

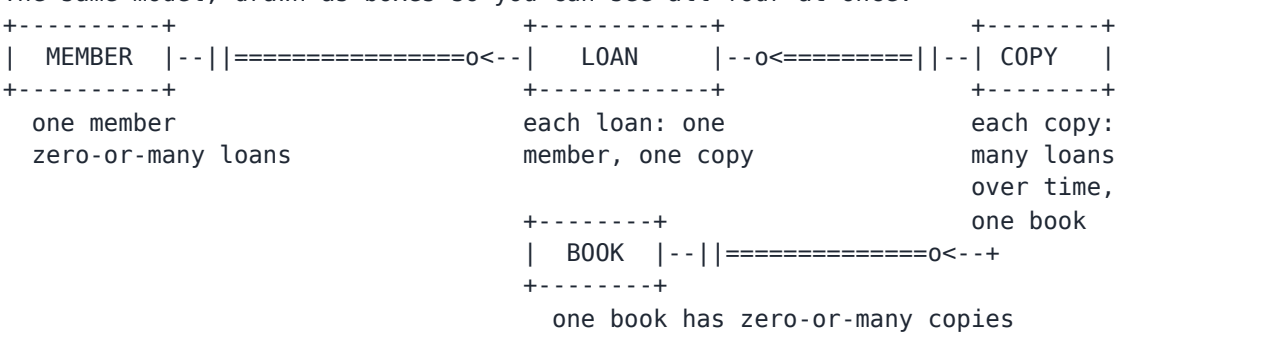
Notice LOAN sits at the "many" end of two 1:N links - it is the bridge between MEMBER and COPY, and it carries its own facts (the dates). It earned its place.

STEP 5 - KEYS. Surrogate id on each table (member_id, book_id, copy_id, loan_id).

Keep email UNIQUE on member and isbn UNIQUE on book as natural identifiers.

STEP 6 - DRAW IT (in text):
Member ||---o< Loan >o---|| Copy >o---|| Book
Read the chain: a member has zero-or-many loans; each loan has exactly one copy; a copy has zero-or-many loans and belongs to exactly one book; a book has zero-or-many copies. Every sentence is true of a real library - the design holds.

The same model, drawn as boxes so you can see all four at once:



Read aloud: MEMBER ||--o< LOAN (one member, many loans); LOAN >o--|| COPY (each loan one copy, a copy many loans); COPY >o--|| BOOK (each copy one book, a book many copies). All true -> keep.

STEP 7 - HARD PATTERNS. The Member-Copy many-to-many (a member borrows many copies over time; a copy is borrowed by many members) is already resolved: LOAN is its bridge, and it holds the pairing's own facts (the dates). Nothing else to resolve - no self-reference, no ternary here.

STEP 8 - NORMALISE. Check "the key, the whole key, nothing but the key":
Loan holds copy_id, member_id, and dates - all facts about THAT loan: good.
The member's name is NOT on Loan (it depends on the member, not the loan): good.
The book's title is NOT on Copy (it depends on the book): good. We are at 3NF.

STEP 9 - THE FINAL TABLES:
member(member_id PK, given_name, family_name, email UNIQUE, joined_on)
book(book_id PK, title, author, isbn UNIQUE)
copy(copy_id PK, book_id FK->book, shelf_code, acquired_on)
loan(loan_id PK, copy_id FK->copy, member_id FK->member,
borrowed_on, due_on, returned_on NULL)

STEP 9b - THE FINAL TABLES, WITH SAMPLE DATA. Here is the whole little library as rows. The ids are consistent across tables, so you can trace any link by hand. (Pretend "today" is 2026-10-05 for the overdue question later.)

member

member_id	given_name	family_name	email (UNIQUE)	joined_on
M-1001	Sara	Adib	sara@example.org	2025-01-10
M-1002	Omar	Reyes	omar@example.org	2025-03-22
M-1003	Mina	Lee	mina@example.org	2026-09-30

(M-1003 Mina has joined but never borrowed - and that is perfectly storable.)

book

book_id	title	author	isbn (UNIQUE)
B-1	Dune	Frank Herbert	9780441013593
B-2	Emma	Jane Austen	9780141439587

```
  | B-3    | The Hobbit  | J.R.R. Tolkien   | 9780261103344 |
  +--------+-------------+------------------+---------------+


  copy  (book_id FK -> book)
  +--------+--------+------------+------------+
  | copy_id| book_id| shelf_code | acquired_on|
  +--------+--------+------------+------------+
  | C-1    | B-1    | A1         | 2024-02-01 |   Dune, copy 1
  | C-2    | B-1    | A1         | 2024-02-01 |   Dune, copy 2
  | C-3    | B-2    | B2         | 2024-05-11 |   Emma
  | C-4    | B-3    | C3         | 2023-11-20 |   Hobbit, copy 1
  | C-5    | B-3    | C3         | 2025-06-30 |   Hobbit, copy 2
  +--------+--------+------------+------------+

  loan  (copy_id FK -> copy, member_id FK -> member; today = 2026-10-05)
  +--------+--------+----------+------------+------------+-------------+
  | loan_id| copy_id| member_id| borrowed_on| due_on     | returned_on |
  +--------+--------+----------+------------+------------+-------------+
  | L-1    | C-2    | M-1001   | 2026-09-30 | 2026-10-14 | (null = out)|
  | L-2    | C-3    | M-1001   | 2026-09-20 | 2026-10-04 | (null = out)|
  | L-3    | C-4    | M-1002   | 2026-09-29 | 2026-10-13 | (null = out)|
  | L-4    | C-1    | M-1002   | 2026-08-01 | 2026-08-15 | 2026-08-10  |
  | L-5    | C-5    | M-1001   | 2026-07-01 | 2026-07-15 | 2026-07-10  |
  +--------+--------+----------+------------+------------+-------------+
  returned_on NULL = still out (L-1, L-2, L-3). The other two are history.

STEP 10 - REVIEW against Step 1's questions:
  "Books this member has out now?" loans where member_id = ? and returned_on is
    NULL, joined to copy and book. Answerable.
  "Is a copy of this title available?" copies of the book with no open loan.
    Answerable. "Who is overdue?" open loans with due_on in the past. Answerable.
  "How many times borrowed?" count loans joined to this book. Answerable. It
  passes - and every table traces back to a real question. That is a finished
  small design, built by the recipe. student-panel is this same method, larger.


THE DESIGN AT WORK - the four questions, ANSWERED by hand against the rows above.
This is the payoff: watch the links do real work.

  Q "Which books does Sara (M-1001) have out now?"
    -> loan rows with member_id = M-1001 AND returned_on IS NULL:  L-1, L-2.
       L-1.copy_id = C-2 -> copy C-2.book_id = B-1 -> book "Dune".
       L-2.copy_id = C-3 -> copy C-3.book_id = B-2 -> book "Emma".
    ANSWER: Sara has Dune and Emma out. (Her old Hobbit loan L-5 is returned, so
    it does not count - returned_on is filled in.)

  Q "Is any copy of The Hobbit (B-3) available right now?"
    -> copies of B-3: C-4 and C-5.
       C-4 has an OPEN loan (L-3, returned_on NULL) -> C-4 is out.
       C-5 has no open loan (its only loan L-5 is returned) -> C-5 is free.
    ANSWER: yes - copy C-5 is available.

  Q "Who is overdue?" (today = 2026-10-05)
    -> open loans (returned_on NULL) whose due_on is before today:
       L-1 due 2026-10-14 -> future, fine.
       L-2 due 2026-10-04 -> PAST -> overdue. member M-1001 = Sara.
       L-3 due 2026-10-13 -> future, fine.
    ANSWER: Sara is overdue (Emma, due 2026-10-04).

  Q "How many times has Dune (B-1) been borrowed, ever?"
```

```
      -> copies of B-1: C-1, C-2. Count ALL loans (open or returned) on those:
         C-2 -> L-1 ;  C-1 -> L-4.
      ANSWER: 2 times. (Note we count history too - the returned L-4 still counts,
      because the question is "ever," not "now." Different question, different
      filter, SAME tables. That flexibility is the reward for a clean design.)


--------------------------------------------------------------------------------
  PART 9 - DESIGN JUDGEMENT: THE DECISIONS WITH NO SINGLE RIGHT ANSWER
--------------------------------------------------------------------------------

The recipe gives you a correct design. Judgement makes it a GOOD one. These are the
recurring trade-offs - the "what could we do differently, and should this exist at
all?" layer. None has one universal answer; each has a question to ask yourself.

  DERIVE vs STORE. If a fact can be computed from others (a total, an age, a count),
    prefer to compute it when asked, so it can never fall out of step. Store it only
    when computing it is genuinely too slow AND you accept the duty to keep it
    fresh. Ask: "can this contradict the facts it is derived from?" If yes, don't
    store it.

  THE FROZEN-FACT EXCEPTION - the one time you DO store a copy. A value you capture
    ON PURPOSE at a moment in time is not a derived value; it is a historical fact
    that must never change, even when the "live" value does. The classic case is
    the price on a sale: the product's price today may be 1.50, but the line of a
    sale from last year must forever say what was CHARGED then.

    Drawn - the PRODUCT price can change; the ORDER_LINE price is frozen:
    PRODUCT (price today)              ORDER_LINE (price AS CHARGED, frozen)
    +--------+----------+--------+     +---------+--------+-----+------------------+
    | prod_id| name     | price  |     | order_id| prod_id| qty | unit_price_at_sale|
    +--------+----------+--------+     +---------+--------+-----+------------------+
    | PR-1   | Mug      | 9.00   |     | O-500   | PR-1   | 2   | 7.50  (old price)|
    | PR-2   | Notebook | 4.00   |     | O-500   | PR-2   | 1   | 4.00              |
    +--------+----------+--------+     | O-700   | PR-1   | 1   | 9.00  (new price)|
       Mug is 9.00 TODAY...            +---------+--------+-----+------------------+
       ...but order O-500 charged 7.50 for it last season, and must keep saying so.
    unit_price_at_sale is NOT a 3NF violation and NOT lazy denormalisation: it is a
    SEPARATE fact ("the price charged on this line") that merely resembles today's
    price. The ORDER TOTAL, by contrast, IS derived - sum of qty*unit_price_at_sale
    across the lines - so you compute it, you do NOT store it.
    (student-panel parallel: a recorded SCORE is a frozen fact of its period - what
    was recorded then stays what was recorded then, even if parameters later change.)


  WHAT NULL MEANS. An empty (NULL) value is not zero and not "" - it is "unknown or
    not-applicable." Before you allow a column to be empty, decide what its
    emptiness will MEAN (loan.returned_on being NULL means "still out"). A NULL you
    can't explain in words is a design you haven't finished thinking about.

    Decide this per column, and write the meaning down:
    +----------------------+-------------+--------------------------------------+
    | column               | NULL allowed| if NULL, it MEANS...                  |
    +----------------------+-------------+--------------------------------------+
    | loan.returned_on     | yes         | "not yet returned - still out"       |
    | member.email         | no (NOT NULL)| (blank email would be nonsense)     |
    | member.middle_name   | yes         | "has no middle name / not recorded"  |
    | employee.manager_id  | yes         | "this is the root - reports to none" |
    | order_line.order_id  | no (NOT NULL)| (a line with no order cannot exist) |
    +----------------------+-------------+--------------------------------------+
    Where blank would be nonsense, forbid it with NOT NULL. Where blank carries a
```

real meaning, allow it - and record what the meaning is, as above.

SOFT vs HARD DELETE. Deleting a row really removes it (hard). Sometimes you instead mark it "deleted_at = now" and hide it (soft delete), so history and references survive and mistakes are recoverable. Soft delete keeps the past but complicates every query ("...and not deleted"). Ask: "will anyone ever need to know this existed?" student-panel soft-deletes students and archives parameters for exactly this reason.

Drawn - a soft-delete column; the row stays, but queries hide it:

student_id	name	is_active	archived_at	
STU-0001	Sara Adib	true	(null)	shown
STU-0002	Omar Reyes	true	(null)	shown
STU-0003	Mina Lee	false	2026-09-15	HIDDEN

Mina's row is not gone - her old scores still point at a real student, and she can be restored. The cost: every normal query must add "WHERE is_active" or it will show her. Hard delete is simpler but forgets forever. Choose on purpose.

A FLEXIBLE (JSON) COLUMN vs PROPER COLUMNS. Most engines let one column hold a free-form bag of data (JSON). It is tempting for "extra, varies-per-row" fields. But data in there can't be checked, keyed, or easily searched by the database. Rule of thumb: facts you will filter, sort, or enforce deserve real columns; only genuinely open-ended extras belong in a flexible column. Reach for it rarely and on purpose.

the fact	real column?	why
student birthdate	REAL column	you sort/filter by it
order status	REAL column	you filter by it; rules
which colour a user picked	REAL column	small, fixed, queryable
arbitrary per-import notes	JSON column	open-ended, never queried, varies wildly

NAMING. Consistent, boring names are a feature: singular-or-plural table names (pick one), id for the primary key, thing_id for a foreign key, lower_snake_case throughout. Future-you reads the schema far more often than you write it.

AVOID RESERVED WORDS. Database engines reserve words like order, group, user, select, table for their own grammar. Name a table one of these and every query needs awkward quoting - or breaks. Pick a safe synonym or a plural. student-panel runs on PostgreSQL, where exactly these words bite:

tempting name	safe name	note
order	orders	"order" is reserved (ORDER BY)
group	groups	"group" is reserved (GROUP BY)
user	app_user / users	"user" is reserved in Postgres
class	classes / cohort	reserved in some engines
select, table	(never)	always reserved - rename the idea

The habit: plural table names (orders, groups, students) sidestep most reserved words for free, and read naturally ("rows in students").

KEY IDEA: correctness is teachable in ten steps; judgement grows by making these calls, stating your reason, and later seeing how it played out. That is exactly

what our compare-against-the-reference loop is for.

PART 10 - PRACTICE (do these before we meet again)

Reading built the vocabulary; only designing builds the skill. Take a blank sheet and run the full ten-step recipe on ONE of these. Don't aim for perfect - aim for reasons you can defend.

DESIGN-FROM-SCRATCH (pick one):

A. A gym: members, membership plans, classes, and which members booked which class sessions. (Watch for a many-to-many and a bridge that carries a "booked_on" date.)

B. A recipe app: recipes, ingredients, and how much of each ingredient a recipe uses. (The quantity is a fact about the recipe-and-ingredient PAIR - where does it go?)

C. A simple ticketing system: customers, tickets, and replies on a ticket. (Look for a weak entity and a possible self-reference among replies.)

STARTER HINTS (a scaffold, NOT the answer). For each, here are the entities you will probably land on and the ONE move that trips people up. Fill in the keys, attributes, and crow's-foot ends yourself - that is the exercise.

ex.	likely tables (yours may differ)	the tricky move to get right
A	members, plans, classes, class_sessions, bookings	member<->class is M:N -> a BOOKINGS bridge; booked_on lives ON the bridge, not on either side
B	recipes, ingredients, recipe_ingredients	quantity belongs to the PAIR -> put it on the recipe_ingredients bridge (amount + unit columns)
C	customers, tickets, replies	a reply is WEAK (needs a ticket); reply.parent_reply_id self-refs for threads, NULL at top level

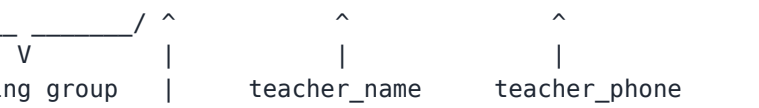
Leave the attribute lists and the min/max marks blank above on purpose; drawing them is what proves you can do it. If you get stuck, re-read Part 5 (the five hard patterns) - every tricky move here is one of those five.

CRITIQUE-A-DESIGN:

D. Someone models a school as ONE table:
(student_name, class1, class2, class3, teacher_name, teacher_phone)
Name every problem you can using Parts 2, 5, and 6 - the repeating class columns, the multi-valued smell, the teacher facts in the wrong place - then redraw it properly. Write WHY each change removes an anomaly.

Here is that bad table drawn out with sample rows, so the trouble is visible. Study it, name each flaw, THEN redraw it on your own sheet (don't peek for a fixed version here - there isn't one; producing it is the task):

student_name	class1	class2	class3	teacher_name	teacher_phone
Sara Adib	Math	Art	PE	Ms. Flores	555-0101
Omar Reyes	Math	Music		Ms. Flores	555-0101
Mina Lee	Science			Mr. Dhillon	555-0199



```
                    (1NF break): 3    |     repeats; its     repeats WITH the
                    class slots, no   |     phone is a fact   name -> if Flores
                    room for a 4th,   |     ABOUT the teacher,changes phone you
                    empties when a    |     not the student   must edit EVERY
                    student takes <3  |     (wrong place)      matching row
                                      |
                              empty slots = the multi-valued smell:
                              "classes" is really many values per student
```

NAME THESE ANOMALIES (your write-up should hit all three):

- UPDATE : change Ms. Flores's phone -> many rows to fix; miss one and the table now disagrees with itself about one real-world fact.
- INSERT : hired a teacher who has no students yet? There is no row to put them in (a row needs a student) -> you cannot record them at all.
- DELETE : remove Mina Lee, the only student linked to Mr. Dhillon, and his phone number disappears with her -> you lost a fact you still need.

The redraw you owe me: students, classes, an ENROLMENTS bridge (student<->class is M:N), and teachers - with teacher_phone living on teachers exactly once. Draw the crow's-foot ends and keys yourself; that drawing IS the exercise.

FOR EACH, KEEP: your purpose sentences and question list, your ER sketch with crow's-foot ends, your final tables with keys, and - most valuable - a note of any decision you were UNSURE about. Those uncertainties are precisely what we'll talk through together.

SELF-CHECK RUBRIC (run your design through this before we meet). If you can tick every row with a reason, you have done the work; any row you cannot tick is a great thing to bring to the table.

ask yourself	pass looks like	re-read which part
Does every cell hold ONE value (no "A, B")?	yes, atomic	Part 6 (1NF)
Any repeating columns (class1, class2 ...)?	none	Part 6 (1NF)
Does each table have a primary key?	one per table	Part 3
Does every M:N have a bridge table?	yes	Part 5
Does each fact live in exactly ONE place?	yes	Part 6 (2/3NF)
Is each FK on the MANY side, pointing at a real row (referential integrity)?	yes	Part 3 / 4
Did you decide ON DELETE for each FK?	yes, per FK	Part 8
Any stored value that could contradict the facts it is derived from?	none (or a frozen fact)	Part 9
Did you note every decision you were unsure about?	yes - bring them!	(all)

WHERE THIS LEADS

Once this method is yours, we point it straight at your real goal: we will run these same ten steps to RE-DERIVE the student-panel schema from a blank page, and watch it arrive (mostly) at the reference design - with every difference a conversation. After that, your questions from the read-through will have a frame to land in. No rush. Bring your sheets.

GLOSSARY - quick reference

ENTITY (TYPE)	a kind of thing you keep facts about; becomes a TABLE.
ENTITY INSTANCE	one specific such thing; becomes a ROW.
ATTRIBUTE	a fact about an entity; becomes a COLUMN.
atomic	holds one indivisible value.
composite	really several values (address -> street, city, postcode).

```
  multi-valued    has many values at once -> needs its own rows/table.
  derived         computable from other facts -> prefer not to store.
RELATIONSHIP      an association between entities; the VERB. Degree = how many
                  entity types it joins (unary/binary/ternary).
CARDINALITY       how many relate to how many: 1:1, 1:N, M:N (the maximum side).
PARTICIPATION     the minimum side: is zero allowed (optional) or required
                  (mandatory)?
CANDIDATE KEY     any attribute set that uniquely identifies a row.
PRIMARY KEY       the one candidate chosen as the official identifier; unique,
                  never empty, ideally never changing.
NATURAL KEY       a real-world identifier (ISBN, email).
SURROGATE KEY     a meaningless invented identifier (auto-number, UUID).
COMPOSITE KEY     a key made of more than one column.
FOREIGN KEY       a column holding another table's primary-key value; a pointer.
REFERENTIAL       the database's guarantee that a foreign key points at a real
  INTEGRITY       row (or is empty, if optional).
BRIDGE / JUNCTION /  a table in the middle that resolves a many-to-many into two
  ASSOCIATIVE table    one-to-manys; holds the pairing's own facts.
WEAK ENTITY       a thing that cannot exist without its parent (order-line).
SELF-REFERENCE    a foreign key pointing back into the same table (parent_id);
                  builds trees/hierarchies of any depth.
NORMALISATION     organising tables so each fact lives in exactly one place.
  1NF             atomic cells, no repeating columns.
  2NF             no non-key fact depends on only PART of a composite key.
  3NF             no non-key fact depends on another non-key fact.
ANOMALY           an update/insert/delete problem caused by redundancy.
DENORMALISE       deliberately keep some redundancy for read speed (with care).
SOFT DELETE       mark a row deleted (deleted_at) instead of removing it.
NULL              "unknown or not-applicable" - decide what each NULL MEANS.
CONCEPTUAL /      the three design heights: ideas -> tables -> real storage.
  LOGICAL / PHYSICAL
FROZEN FACT       a value copied ON PURPOSE at a moment and never changed again
                  (unit_price_at_sale; a recorded score). NOT the same as a
                  derived value and NOT denormalisation - it is history.
UNIQUE            a constraint: no two rows may share this value (student_code,
  (CONSTRAINT)    email). Guards natural keys even when the PK is a surrogate.
NOT NULL          a constraint: this column must always hold a value.
ON DELETE         what happens to child rows when their parent is deleted:
  RESTRICT        refuse the delete while children exist (the safe default).
  CASCADE         delete the children too (use only when they can't outlive it).
  SET NULL        keep the children but empty their foreign key (must be optional)
ORPHAN            a child row whose foreign key points at a parent that no longer
                  exists; referential integrity exists to prevent these.
RESERVED WORD     a word the database grammar owns (order, group, user, select);
                  don't name tables/columns these - use orders, groups, app_user.
HARD DELETE       actually remove the row (opposite of SOFT DELETE); frees space
                  but loses history and can orphan children - choose per table.

================================================================================
  END. Next: bring your practice sheets; we re-derive student-panel with this
  same ten-step method, then your questions.
================================================================================
```